
CACHE COHERENCE

Cache Coherence

Topics:

- Why coherence is required
- What goes wrong without it
- Hardware solutions
- MESI protocol
- Impact on system performance

From Single Core to Multicore

Previously:

- Single core -> one cache -> no problem

Now:

- Multiple cores each with private caches

Problem:

- Multiple copies of the same data

The Core Problem

Example:

Core 1: $X = 10$ (in cache)

Core 2: $X = 10$ (in cache)

Core 1 updates $X = 20$

What does Core 2 see?

Without coherence Core 1 sees $X = 20$ and Core 2 sees $X = 10$

Inconsistent system state that breaks program correctness

Definition of Cache Coherence

A system is coherent if:

1. A read returns the most recent write
2. Writes to the same location are seen in the same order by all the processors

Concept	Meaning
Coherence	Same value per location
Consistency	Ordering across locations

Two Possible Solutions

To maintain coherence::

1. Write-invalidate:
 - Invalidate other copies
2. Write-update:
 - Propagate updates

Modern systems use invalidate protocols

Snooping Protocols

Basic idea: all caches monitor a shared bus and observe other cores' memory operations

When a core writes the others react accordingly

Each cache line has:

- State (coherence state)
- Tag
- Data

Transition occur based on local actions (read/write) and bus actions (other cores)

Introducing MESI Protocol

The MESI invalidation protocol defines four states:

- M -> Modified
 - Only copy
 - Dirty (different from memory)
- E -> Exclusive
 - Only copy
 - Clean
- S -> Shared
 - Multiple Copies
 - Clean
- I -> Invalid
 - Not present

Each cache line is always in one of these states

State Transitions

Operations:

- Read hit
- Read miss
- Write hit
- Write miss

Plus:

- Bus read
- Bus write

Each triggers state changes

Example

Core 1 reads X

- Not in cache -> miss
- Fetch from memory

If no other core has it -> state = Exclusive

Core 2 reads X but Core 1 already has X

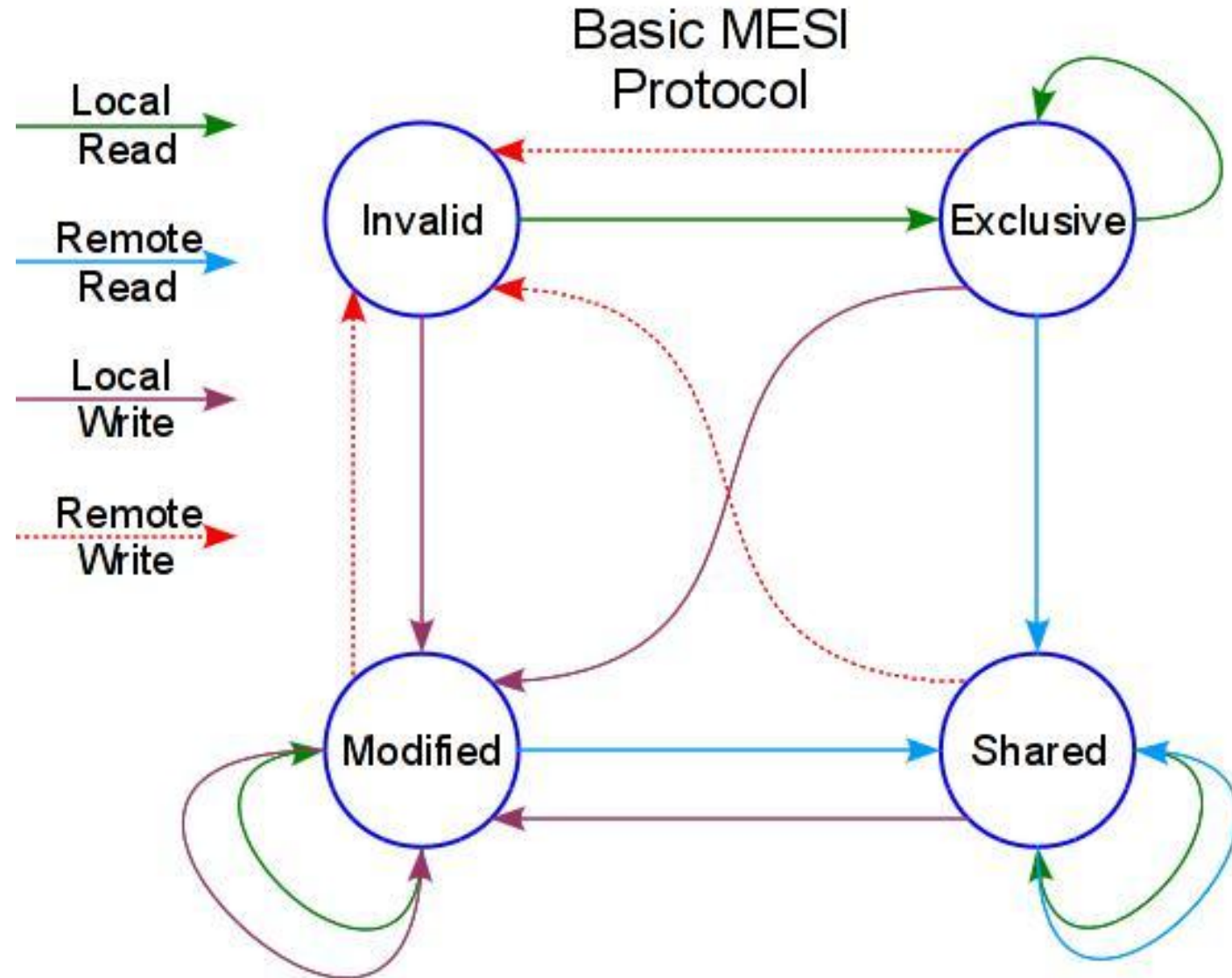
Result -> both cores state = Shared

Core 1 writes X (must invalidate other copies!)

Result -> Core 1 state = Modified and Core 2 state = Invalid

Core 1 writes X again -> No bus traffic needed, only local update

MESI State Diagram



Why invalidate Protocols Win

Write-updates have high bandwidth cost

Write invalidate:

- Fewer messages
- Scalable
- Simpler

Almost all modern CPUs use invalidate protocols

With respect to single core CPUs, coherence introduces extra traffic, invalidations, stalls and latency variability

False Sharing

Multiple cores access different variables that happen to reside in the same cache line and at least one core writes

Data are not logically shared, but the hardware sees them as shared

Reason -> cache works at cache line level, not at single variable level

Example:

- cache with 4 B blocks

- Variable A at address 0xABCD0000

- Variable B at address 0xABCD0001

The two variables are independent but mapped on the same cache line

False Sharing

Core 1 caches A

Core 2 caches B

both cores load a block X of 4 B containing both A and B
the state of the block X is shared

Core 1 writes A -> write hit in cache A

X is Modified in cache A

X is Invalid in cache B

Core 2 writes B -> since the block is invalid it caches the new data

X in is Invalid in cache A

X is Modified in cache B

False Sharing

What happens if Core 1 writes A again?

The result is that the operations of the previous slide are repeated

There is a ping-pong of the cache line between the two cores
continuous invalidation, high traffic and huge latency

It is defined “false” because there are not shared variables between the cores, but the hardware is not aware of this

False Sharing - Performance

False sharing impacts on performance with

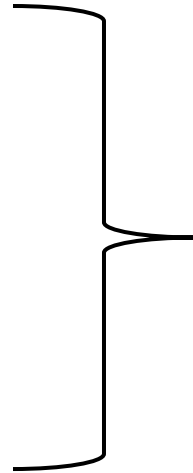
- Continuous invalidations
 - Each writes Invalidates the other cores
- Cache lines bouncing
 - A line continuously switches between the cores
- High coherence traffic
 - The communication bus could be saturated
- Bad scalability
 - Adding cores degrades performance

False Sharing – Example

Int counter[2];

Thread 1:
counter[0]++;

Thread 2:
counter[1]++;



counter1 and counter2 are in the same cache line

False Sharing - Performance

How to solve?

- Padding -> separate variables to use different cache lines

```
struct {  
    int value;  
    char padding[60];  
} counter1, counter2;
```

- Align to cache line
 __attribute__((aligned(64)))
- Data layout redesign

Engineering Impact

Coherence affects:

- Scalability
- Latency
- Bandwidth
- Power

In large systems coherence becomes dominant

Closing

Key concepts:

- Why coherence is necessary
- MESI protocol
- Invalidate mechanism
- False sharing

Next episode: memory consistency models (ordering constraints, TSO, weak models)